

# Scheduling a Single AGV to Transport Identical Jobs in a Flow Shop without Buffer

Lukas Krajcik

*Department of Advanced Computing Sciences*

*Faculty of Science and Engineering*

*Maastricht University*

*Maastricht, The Netherlands*

**Abstract**—This paper studies a flow-shop with  $n$  identical jobs that must be processed on  $m$  stations in a specific order. The jobs are transported between stations via a single Automated Guided Vehicle (AGV). Before the job is transported from station  $i$  it first must be processed there, lasting time  $p_i$ . The AGV can transport one job at a time, and cannot store jobs anywhere but in an empty station. We study the problem of computing schedules for the AGV that minimize the makespan. We first analyze a trivial schedule that transports a job through all stations before transporting other jobs, showing it is almost optimal under certain assumptions. We then propose an algorithm to find an optimal schedule in constant time with respect to the number of jobs when there are two stations. The idea is extended to design a schedule for a flow-shop system with an arbitrary number of stations. For this, we study so-called cycles of schedules and use the derived properties to characterize certain class of optimum schedules. We prove that the designed schedules have an asymptotic approximation guarantee of  $(1 + \varepsilon)$  for any  $\varepsilon > 0$ .

## I. INTRODUCTION

In recent years, transportation robots such as automated guided vehicles (AGV) have become increasingly popular in the manufacturing context. AGVs are used to improve the cost and efficiency of manufacturing processes. Conventionally, such a process constitutes a fixed assembly line with multiple stations on which the job, an arbitrary product, is being processed and transported via a conveyor belt. Different jobs may require different assembly lines, which is not possible with a fixed conveyor belt. A flexible assembly line offers this flexibility, where an AGV facilitates flexible movement between stations, allowing specific jobs to follow their specific assembly lines. At any time, an AGV must decide which job to transport next, which creates many possible schedules for an AGV to follow. A schedule of an AGV has an impact on aspects such as completion time of jobs or energy consumption. These aspects affect the cost of production and are therefore important in any real manufacturing plant. There are a multitude of real-world scenarios in many industries, including textile [1], paper [2], and electronics production [3], where flexible assembly lines are used.

The field of scheduling jobs in flexible assembly lines is vast, with a large number of configurable parameters,

elaborated it [4]. The main ones are the processing times, types and amounts of machines on different stations, transportation costs, jobs with restrictions on processing, and more. Finally, there are different optimality measures to consider, where the most common one is the makespan, the completion time of the last job at the last station. Other optimality measures include idle time, energy consumption, and more.

The purpose of this paper is to study a restricted case of a flexible assembly line and to design efficient schedules with respect to (w.r.t.) makespan, along with providing a theoretical understanding of the problem. We assume that the flexible assembly system consists of equivalent jobs  $n$  in an initial pick-up station. All  $n$  jobs must be processed in  $m$  stations in a specific order, where each station has exactly one machine to process exactly one job at a time. Each job, after being processed at all  $m$  stations, must be transported to an unloading station where it is stored. There is only one AGV, and it can transport exactly one job at a time. None of the stations have buffers, meaning that a job can only be transported to the next station if there is no job at that station. Since all  $n$  jobs are equivalent, the order in which the jobs are processed in the system is not considered. The rest of the configuration, such as the number of stations, jobs, or the processing and transportation times, can vary. The only problem to solve is to find a schedule for the AGV which has the lowest makespan – an optimal schedule.

We formalize the problem in Sections III, IV. Next, we design a simple schedule and compare it with an optimal schedule under various assumptions (Section V). Since this schedule does not prove to be efficient, we investigate the cyclic behavior of schedules in Section VI. The cyclic behavior will allow us to find a trivial optimal schedule for any two-station system (Section VII). The concepts are then applied to any  $m$ -station system, where we design a schedule that approaches an optimal schedule in makespan for large  $n$  (Section VIII). In Sections IX, X, we discuss the results and draw conclusions.

## II. RELATED WORK

There are many configurations of a flexible manufacturing environment that affect the scheduling aspects and restrictions that need to be considered. Among the environments studied are the hybrid flow-shop [4], job-shop [5], flexible flow-shop

This thesis was prepared in partial fulfillment of the requirements for the Degree of Bachelor of Science in Data Science and Artificial Intelligence, Maastricht University. Supervisor(s): [Matus Mihalak, Steven Kelk]

[6], where each of them differs in the production paths jobs must take, the types of stations and machines on which the jobs are processed, precedence constraints, and more. The literature studying these environments does not, in general, consider costs associated with transporting jobs between stations. Finally, there are multiple optimization tasks to consider such as tardiness, the completion time of all jobs, idle time, and more. The authors of [4] survey many methods and configurations of the hybrid flow-shop environment, showing that one of the most popular algorithms to solve problems in this environment is the B&B method, along with heuristics for lower and upper bounding.

Extensive literature exists on the scheduling of jobs without transportation constraints, for instance, the flow-shop problem, but also AGV routing and scheduling. However, research which concurrently considers both aspects is more limited.

Our problem is in a sense extension of the flow-shop problem as defined in [7], with the addition of transportation times. A robotic cell problem studied in [8] is characterized as a flow-shop with no buffers, one AGV with capacity one, and different types of jobs. The problem we study is a specialization of the robotic cell problem since we also assume all jobs to be identical. In [9] the exact setting as we study in this paper is considered. The authors characterize the feasibility of any schedule using linear constraints, which can be exploited to formulate the problem as an Integer Linear Program (ILP). In its raw form, the ILP requires hours of computation time, even for the configuration of 6 stations and 25 jobs. Therefore, they propose a time-window heuristic, where the original schedule is decomposed into smaller schedules which are then joined to create the complete schedule. With this heuristic, they can find schedules whose makespan is as close as 2% to the minimum makespan, given 7 jobs and 5 stations. For larger instances, the exact performance is unknown since it is infeasible to compute the minimum makespan within a reasonable time frame.

Authors of [10], [11], propose genetic programming along with the B&B method to solve a generalization of our problem, where different types of jobs can appear. The results show that the B&B approach can solve smaller instances, 14 jobs, and 5 machines, in under 1 hour. For 5 machines and 16 or more jobs, the solution is not found within the 1-hour limit.

Of course, these results are not directly linked to our problem, since they propose methods that are exponential in time complexity, or do not investigate the properties of specific schedules directly, relying on experimental evaluation.

The theoretical properties of our problem are unknown; however, it was proved that for a two-station problem with unlimited buffers, the problem is NP-Hard [12]. The problem is NP-hard even without transportation costs, three or more stations, and different types of jobs, as proven in [13]. The authors of [14] prove that the robotic cell problem is NP-Hard for two stations and sequence-dependent setup times. These results do not address the problem we study, since they consider a different, or more general setting than us. Therefore, the complexity of our problem is unknown.

To our knowledge, there are no papers that directly study

and design schedules. Instead, most of the work is focused on designing algorithms to produce schedules. The algorithms can be either exact but exponential in terms of the number of jobs, or not well-bounded, relying on statistical evaluation. They mostly utilize methods such as linear programming or metaheuristics, not exploiting properties specific to the narrow problem studied. Consequently, it is unknown whether there are schedules with specific structures that are easy to find, and if their makespan is a well-behaved approximation of the lowest possible makespan.

### III. PROBLEM DEFINITION

We assume there are  $n$  identical jobs and  $m + 2$  stations. The first and last stations denote the pick-up station where all the jobs are initially stored, and the unloading station, where all the jobs are stored at the end. Every job must be processed on stations  $1, \dots, m$  in a specific order  $-(1, \dots, m)$ . Due to the equivalency of all jobs, the order of the jobs in the pick-up station can be arbitrary, which was elaborated in [9]. The transportation time (or cost, equivalently) is defined between every pair of stations  $i$  and  $j$ , denoted by  $d_{i,j}$ . We assume the transportation time between any two stations to be the same irrespective of whether the AGV is or is not carrying a job. The AGV can transport only one job at a time, and every station apart from the pick-up and unloading stations can process and store exactly one job at a time. A job can be transported from station  $i$  to station  $i + 1$  only if station  $i + 1$  is empty. In addition, a job cannot be held in buffers between the stations. When a job is delivered to a station  $i$ , its processing starts immediately and lasts time  $p_i$ . If a job is at station  $m + 1$  – the unloading station, it is effectively out of the system and does not need to be considered further. For simplicity, we assume all quantities defined in the system to be positive integers. We refer to the aforementioned scenario as an  $m$ -station system, where  $m + 2$  is the number of stations, including the pick-up and unloading station.

The problem is to find a schedule for the AGV, so that the AGV delivers all  $n$  jobs into the unloading station in the lowest makespan while ensuring each job is processed in the specific order on the  $m$  stations. We refer to this schedule as an optimal schedule ( $OS$ ).

### IV. SCHEDULE DEFINITION

At any time, the flow-shop system can be described using a state triplet  $L|P|loc$ , where:

- $L = (b_1, b_2, \dots, b_m)$  is a tuple where each  $b_i \in \{0, 1\}$ . Each  $b_i$  states whether a job is ( $b_i = 1$ ) or is not ( $b_i = 0$ ) at station  $i$
- $P = (t_1, t_2, \dots, t_m)$  is a tuple where each  $t_i \in [0, p_i]$ . Each  $t_i$  denotes the remaining processing time of a job at station  $i$ .
- $loc \in \{0, 1, \dots, m, m + 1\}$  denotes the station at which the AGV vehicle is

We stress that neither the pick-up nor the unloading station are modeled in a state triplet. Additionally, the remaining

number of jobs in the pick-up station is not modeled in a state triplet. See Figure 1 for an example of a state sketch.

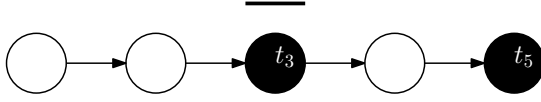


Fig. 1: Depiction of state for a system with 7 stations, pick-up and unloading stations not depicted. A black circle depicts a job at that station with remaining processing time  $\geq 0$ . The thick black line indicates the location of an AGV. Variables  $t_3$  and  $t_5$  denote the remaining processing times.

In [9], a schedule is defined as a sequence of transportation tasks the AGV must execute. Each transportation task corresponds to exactly one job being transported from station  $i$  to station  $i + 1$  in the shortest time possible. Consider the flow-shop system to be at state  $L|P|loc$ , where  $b_i = 1, b_{i+1} = 0$  and  $t_i = q$ . Therefore, a transportation task that transports the job from station  $i$  to station  $i + 1$  can be executed. The time cost of a transportation task consists of three parts. First, the AGV travels  $d_{loc,i}$  time to reach station  $i$ . Next, it must wait  $\max(0, q - d_{loc,i})$  time, because the job at station  $i$  may not be processed yet. Finally, the AGV transports the job from station  $i$  to station  $i + 1$  which takes  $d_{i,i+1}$  time. With this, the transportation task ends, and the state at which the flow-shop system is at this point is such that  $b_i = 0, b_{i+1} = 1$ , and  $t_{i+1} = p_{i+1}$  because the job just started being processed at station  $i + 1$ .

Any transportation task can be described as an appropriate pair of states, where the first state describes the system before the task is executed, and the second state describes the system after. There is no need for any intermediate states to describe the transportation task since the task is executed in minimum time – minimizing the makespan (see Figure 2)

The authors in [9] show that a schedule consists of  $n \cdot (m + 1)$  transportation tasks. In our formulation, this corresponds to a sequence of  $1 + n \cdot (m + 1)$  states, since each adjacent pair of states corresponds to one task. A schedule is denoted as  $(x_0, x_1, \dots, x_{n \cdot (m + 1)})$ . The first state  $x_0$  is always the state where  $loc = 0$  and  $\forall b_i, t_i : b_i = 0, t_i = 0$ . State at any other index  $i$  must be induced by a transportation task executed on state  $i - 1$ . The state  $x_{n \cdot (m + 1)}$  must be such that,  $\forall b_i, t_i : b_i = 0, t_i = 0, loc = m + 1$ . To ensure all  $n$  jobs were unloaded, there must be  $n$  states inside a schedule such that  $loc = m + 1$ . A sub-schedule is a contiguous subsequence of a schedule of the form  $(x_a, x_{a+1}, \dots, x_{a+k})$ . We sometimes consider a sequence of states  $(x, y, \dots, z)$  that would constitute a valid sub-schedule if they were in a schedule. We refer to such a sequence as a valid sequence. A valid sequence could appear in a schedule, but we have not yet determined a specific schedule in which it appears. A schedule, sub-schedule, or a valid sequence must be such that any state in the sequence but the first one is the result of a transportation task executed on the previous state.

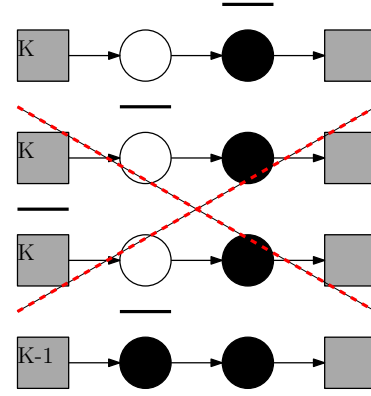


Fig. 2: An example of a transportation task of moving a job from station 0 to station 1. The crossed states are not considered, since they are redundant in information. K denotes the remaining number of jobs at the pick-up station.

An upper bound on the number of feasible states is:

$$(m + 2) \cdot (\max_i p_i)^m \cdot 2^m \quad (1)$$

The quantity (1) is derived by considering all possible configurations of all three parts of a state triplet. Since all parameters are assumed to be integers, if a job is at station  $i$ ,  $t_i$  can be at most  $\max_i p_i$ , and every time increment is an integer, thus  $t_i$  can only be an integer. Therefore, there are at most  $\max_i p_i$  values any  $t_i$  entry can have. Notice that the quantity (1) does not depend on  $n$ , but the length of a schedule does. There is room to improve this upper bound. For instance, if  $b_i = 0$ , there is no job at station  $i$ , hence we can arbitrarily set  $t_i = 0$ , narrowing down the number of unique states.

#### A. Schedule Operations

We define three operators on an arbitrary schedule, sub-schedule, or a valid sequence  $A$  which describes its properties.

- $cost(A)$  is the total time it takes to execute all transportation tasks. Since every adjacent pair in a schedule corresponds to a transportation task, the cost must be additive, i.e.,  $cost(x_i, y_{i+1}, z_{i+2}) = cost(x_i, y_{i+1}) + cost(y_{i+1}, z_{i+2})$
- $jobs(A)$  is the number of jobs that are unloaded in the unloading station during  $A$
- $rate(A) = \frac{cost(A)}{jobs(A)}$ , is the average time spent to unload one job during  $A$

By definition, the makespan of a schedule is the  $cost$  of the schedule. Since the state triplet contains all remaining processing times, and AGV location; a transportation task defined by a specific adjacent pair of states always incurs the same  $cost$ , no matter where, and how many times in the schedule this pair of states appears.

#### V. BOUNDS ON SPECIFIC SCHEDULES

First, we analyze the lower bounds of the optimum schedule. Then, a simple schedule is designed, and its makespan is compared to the devised lower bounds. For simplification, we

assume that the layout of the  $m$ -station system is on a line. With a general layout, the results will differ.

#### A. Lower bounds on optimum makespan

We provide several lower bounds of the makespan of any schedule, thus also the makespan of  $OS$ . Let  $D = \sum_{i=0}^m d_{i,i+1}$  where  $d_{i,i+1}$  is the distance between the stations  $i$  and  $(i+1)$ . Let  $P = \sum_{i=1}^m p_i$ .

Since there is only one AGV, every job must be transported between stations  $i$  and  $i+1$  one by one incurring cost  $d_{i,i+1} n$  times. This must be done for every station  $i$ . Thus we obtain

$$\text{makespan}(OS) \geq n \cdot D, \quad (2)$$

where  $\text{makespan}(OS)$  is the makespan of an optimum schedule. Since the AGV travels from station  $i$  to station  $i+1$   $n$  times, it must also travel from station  $i+1$  back to station  $i$   $n-1$  times, ensuring it will end up station  $m+1$  at the end of a schedule. This is only valid if the system is on a line, resulting in additional cost  $(n-1) \cdot D$ . Thus the makespan of  $OS$  can be bounded as follows:

$$\text{makespan}(OS) \geq 2 \cdot n \cdot D - D \quad (3)$$

Another bound is based on processing time. For any specific station  $i$ ,  $n$  jobs must pass through, and they cannot do so in parallel, because the station can only process one job at a time. This also applies to the station with the largest processing time. Clearly, the average processing time  $P/m$  is smaller or equal to  $\max_i p_i$ , thus can also be used to define another lower bound, now in terms of  $m$ . Therefore we get:

$$\text{makespan}(OS) \geq n \cdot (\max_i p_i) \geq n \cdot (P/m) \quad (4)$$

#### B. The Primitive Schedule

The Primitive Schedule ( $PS$ ) is a schedule in which the AGV moves one job to station 1, then station 2, until the station  $m+1$ , without moving any other job from the pick-up station. Thus, the AGV must wait for time  $p_i$  in each station  $i \in \{1, \dots, m\}$  to transport one job. Additionally, when a job is delivered to the unloading station, the AGV travels  $D$  time to retrieve the next job from the pick-up station. This schedule, trivially, has a makespan

$$2 \cdot n \cdot D - D + n \cdot P. \quad (5)$$

**Theorem 1.** When for every  $i, j, k$ ,  $p_i \leq d_{j,k}$ , then the primitive schedule is an optimal schedule.

*Proof.* Let  $PS$  be the primitive schedule and  $\Gamma$  be any other schedule. In schedule  $\Gamma$ , consider the AGV at an arbitrary moment when the  $k$ -th job has been delivered to station  $i$ , denoting this state as  $\alpha$ . Assume, instead of waiting  $p_i$  time, and transporting  $k$ -th job to station  $i+1$ , the AGV transports another job to station  $j$ . Eventually, the AGV must return to station  $i$  to transport the  $k$ -th job to station  $i+1$ . When the AGV arrives at station  $i$ , the state at which the system is will be denoted as  $\beta$ .

At state  $\alpha$ ,  $k$ th job is at station  $i$ , meaning  $k$  jobs passed through stations  $i-1$ ,  $i$ , and  $k-1$  jobs passed through stations

$i, i+1$ . Therefore, the costs  $x_1 = 2 \cdot (k-1) \cdot d_{i,i+1}$  and  $y_1 = 2 \cdot k \cdot d_{i-1,i}$  were already incurred. At state  $\beta$ ,  $k$ th job is still at station  $i$ , thus the costs  $x_2 = 2 \cdot (n-k+1) \cdot d_{i,i+1} - d_{i,i+1}$ , and  $y_2 = 2 \cdot (n-k) \cdot d_{i-1,i} - d_{i-1,i}$  must still be incurred (to unload the rest of the jobs). Clearly  $x_1 + x_2 = 2 \cdot n \cdot d_{i,i+1} - d_{i,i+1}$  and  $y_1 + y_2 = 2 \cdot n \cdot d_{i-1,i} - d_{i-1,i}$ . The lower bound  $2 \cdot n \cdot D - D$  includes exactly these costs, but not more. However, right after state  $\alpha$  (between states  $\alpha$  and  $\beta$ ), the AGV traveled away from station  $i$ , incurring an immediate additional cost of either  $d_{i-1,i}$  or  $d_{i,i+1}$  (depending on which direction does the AGV move).

To conclude, for any arbitrary moment and  $\alpha$  where, instead of waiting time  $p_i$ , the AGV decides to transport another job, it incurs immediate cost  $d_{a,a+1}$ , saving waiting time  $p_i$ . Therefore, for all such moments in  $\Gamma$  where the AGV, instead of waiting time  $p_i$  as in  $PS$ , moves elsewhere, a cost  $d_{a,a+1}$  is incurred on top of the guaranteed cost  $2 \cdot n \cdot D - D$ .  $\square$

**Theorem 2.** When  $P \leq D$ , primitive schedule is a  $3/2$ -approximation of an optimal schedule

*Proof.* Using the lower bound in (3), the makespan ratio of  $PS$  to  $OS$  is:

$$\frac{\text{makespan}(PS)}{\text{makespan}(OS)} \leq \frac{2nD - D + nP}{2nD - D} \stackrel{P \leq D}{\leq} \frac{2nD - D + nD}{2nD - D} \simeq \frac{3}{2} \quad (6)$$

$\square$

**Theorem 3.** When  $P > D$ , primitive schedule is a  $3m$ -approximation of an optimal schedule.

*Proof.* Using the lower bound in (4):

$$\frac{\text{makespan}(PS)}{\text{makespan}(OS)} \leq \frac{2nD - D + nP}{n(P/m)} \stackrel{P > D}{\leq} \frac{2nP - P + nP}{n(P/m)} = 3m - (m/n) \leq 3m \quad (7)$$

$\square$

It is unknown whether the upper bound  $3m$  is tight. We will show that the tight upper bound on the ratio cannot be smaller than  $m$  and larger than  $m+2$ .

**Theorem 4.** When  $P \geq D$ , the tight upper bound of  $\frac{\text{makespan}(PS)}{\text{makespan}(OS)}$  is in between  $m$  and  $m+2$ .

To narrow the gap, consider the case when  $D = 0$  and all processing times are the same, thus  $\frac{P}{m} = \max_i p_i$ .

Consider the schedule  $SS$  defined below:

- transport all jobs, starting from the rightmost job, one station to the right, and repeat. (This also includes the transportation from station 0 to station 1)

In the schedule  $SS$ , the jobs are first transported onto the  $m$  stations  $m$  times. When all  $m$  machines are occupied, all jobs are transported one to the right  $n$  times, each time unloading one job. Each transportation of all the jobs one to

the right takes at most  $\max_i p_i = \frac{P}{m}$  time, since there are no transportation costs, and after  $\max_i p_i$  all jobs are guaranteed to be processed. Therefore, the upper bound on the makespan of the schedule  $SS$  is  $(m+n) \cdot \frac{P}{m}$ .

The makespan ratio of  $PS$  and  $SS$  is:

$$\frac{\text{makespan}(PS)}{\text{makespan}(SS)} \leq \frac{n \cdot P}{(m+n) \cdot (P/m)} = \frac{n \cdot m}{m+n} \xrightarrow{n \rightarrow \infty} m \quad (8)$$

### C. Reduction of a System

So far, the lower bounds on  $OS$  have been obtained by analyzing an  $m$ -station system directly. We will show how to reduce any  $m$ -station system to a simpler system such that the makespan of  $OS$  in this simple system is guaranteed to be lower (or equal) than the makespan of  $OS$  in the original  $m$ -station system. This provides a valid lower bound on the makespan of  $OS$  in the original  $m$ -station system.

One such reduction is from a  $m$ -station system to a 1-station system. Suppose we choose arbitrary station  $i$  out of the  $m$  stations; for every  $a$  we set  $d_{a,a+1} = 0$ , and for every  $b$  we set  $p_b = 0$  apart from  $d_{i-1,i}, p_i, d_{i,i+1}$ . Clearly,  $OS$  in this adjusted  $m$ -station system has a lower or equal makespan than  $OS$  of the original system. It is easy to check that this system is equivalent to a 1-station system with  $d_{0,1} = d_{i-1,i}, p_1 = p_i, d_{1,2} = d_{i,i+1}$  (because all jobs can travel to station  $i-1$ , and from station  $i+1$  instantaneously in the adjusted  $m$ -station system). In any 1-station system, there is only one possible schedule (the primitive schedule) and its makespan is

$$n \cdot (2 \cdot d_{0,1} + p_1 + 2 \cdot d_{1,2}) - (d_{0,1} + d_{1,2}). \quad (9)$$

Quantity in (9) is therefore a valid lower bound for the makespan of  $OS$  in the original  $m$ -station system.

For any system the following inequality (an average is smaller than the maximum), where we additionally define  $p_0, p_{m+1}, d_{-1,0}, d_{m+1,m+2} = 0$ , holds:

$$\begin{aligned} \frac{P+2D}{m+2} &= \frac{\sum_{i=0}^{m+1} (d_{i-1,i} + p_i + d_{i,i+1})}{m+2} \leq \\ &\leq \max_i (d_{i-1,i} + p_i + d_{i,i+1}) \end{aligned} \quad (10)$$

We use this inequality to simplify the makespan of the reduced 1-station system, making it dependent on the number of stations  $m$  of the original  $m$ -station system.

First, we reduce the original  $m$ -station system, so that all distances and processing times, apart from  $d_{j-1,i}, p_j, d_{j,i+1}$  where  $(d_{j-1,j} + p_j + d_{j,j+1}) = \max_i (d_{i-1,i} + p_i + d_{i,i+1})$ , are zero. This results in a 1-station system with makespan

defined in (9). Next, we substitute the left-hand side of (10) into (9) and simplify:

$$\begin{aligned} &n \cdot (2 \cdot d_{j-1,i} + p_j + 2 \cdot d_{j,j+1}) - (d_{j-1,j} + d_{j,j+1}) \geq \\ &n \cdot \left( \frac{P+2D}{m+2} \right) + n \cdot (d_{j-1,j} + d_{j,j+1}) - (d_{j-1,j} + d_{j,j+1}) \stackrel{n \geq 1}{\geq} \\ &n \cdot (P+2D) \cdot \left( \frac{1}{m+2} \right) \end{aligned} \quad (11)$$

Since (9) is a lower bound on the makespan of  $OS$  of any  $m$ -station system, quantity in (11) also is. We can define a new upper bound on the makespan ratio of  $PS$  and  $OS$ :

$$\begin{aligned} \frac{\text{makespan}(PS)}{\text{makespan}(OS)} &= \frac{n \cdot (P+2D) - D}{n \cdot (P+2D) \cdot \left( \frac{1}{m+2} \right)} \leq \\ &\frac{n \cdot (P+2D)}{n \cdot (P+2D) \cdot \left( \frac{1}{m+2} \right)} = m+2 \end{aligned} \quad (12)$$

*Proof of Theorem 4.* We have found a case where the ratio of  $PS$  and  $OS$  is at least arbitrarily close to  $m$ . Using reductions we have shown an upper bound of  $m+2$  on the ratio. Therefore, the tight upper bound must be at least  $m$  and is at most  $m+2$ .  $\square$

## VI. CYCLES

Since the number of states is constant w.r.t.  $n$ , two identical states must appear in every schedule that unloads a sufficient number of jobs. We call any sub-schedule that starts and ends in the same state a *cycle*. Figure 3 depicts all possible states and transportation tasks between them for a 2-station system. A cycle is a valid sequence of states that starts and ends in the same state, denoted as  $xCx$ , where  $x$  is the outer, initial, and final state and  $C$  is the valid sequence of all the inner states. Optionally, we will refer to the entire cycle  $xCx$  as just  $C^x$  to simplify notation.

**Definition 1. Indexed Cycle** is a cycle that is a specific sub-schedule of a schedule  $\Gamma$ . An indexed cycle has notation  $x_i C_i x$ . The index  $i$  denotes the index in schedule  $\Gamma$  where the cycle begins.

**Definition 2. Pure Cycle**  $xCx$  is a cycle such that  $x$  appears exactly twice in  $xCx$ , and  $\forall z \in C : z$  appears exactly once in  $C$  and  $z \neq x$ . In other words, there is no smaller cycle inside the cycle  $xCx$ .

**Definition 3. Composite Cycle**  $xCx$  is a cycle such that,  $\exists z \in C : z$  appears either more than once in  $C$  or  $z = x$ . Such a cycle is composed of inner, smaller cycles.

**Definition 4. Disjoint Indexed Cycles**

Two indexed cycles  $x_i C_i x$  and  $y_j T_j y$  of a schedule are called disjoint if their corresponding sub-schedules have at most one state of the schedule in common. Thus, they are either completely different sub-schedules, or they share the outer state (thus  $x = y$ ).

**Definition 5. Contraction of a Cycle** on a specific schedule  $\Gamma$  is an operation, where an indexed cycle  $x_i C_i x$  is contracted to just  $x_i$ .

Contracting an arbitrary indexed cycle  $x_i C x$  on an arbitrary schedule  $\Gamma = (J, x_i C x, K)$ , where  $J, K$  are sub-schedules, will result in a sequence of states  $\Gamma^* = (J, x_i, K)$ . Clearly, every pair of adjacent states of  $\Gamma^*$  still corresponds to an existing transportation task. If  $\Gamma$  unloads  $n$  jobs,  $\Gamma^*$  is a valid schedule when  $n - \text{jobs}(x_i C x)$  jobs are to be unloaded.

**Lemma 1.** *For any schedule  $\Gamma$  with two indexed cycles  $x_i C x$ ,  $y_j T y$  inside it (as its sub-schedules), such that  $\text{jobs}(x_i C x) = \text{jobs}(y_j T y)$ ,  $x_i C x \neq y_j T y$ , and  $x_i C x, y_j T y$  are disjoint indexed cycles, there exists a schedule  $\Gamma^*$  such that  $\text{cost}(\Gamma^*) \leq \text{cost}(\Gamma)$ , and in  $\Gamma^*$  one of the cycles  $x_i C x, y_j T y$  is never present.*

*Proof.* Without loss of generality (w.l.o.g) assume  $\text{cost}(x_i C x) \leq \text{cost}(y_j T y)$ . Additionally, w.l.o.g assume cycle  $x_i C x$  appears before  $y_j T y$  in  $\Gamma$ . The same reasoning applies when their order in schedule is switched.

Consider an arbitrary schedule  $\Gamma$  which has both cycles  $x_i C x$  and  $y_j T y$  inside. Since these cycles are disjoint indexed, they can either share one state or share zero states in  $\Gamma$ , creating two separate cases to consider.

**Case 1:**  $\Gamma = (J, x_i C x, K, y_j T y, L)$ , where  $J, K, L$  are arbitrary sub-schedules, and  $\text{jobs}(x_i C x) = \text{jobs}(y_j T y)$ . Then we can create a new schedule  $\Gamma^* = (J, x_i C x, K, y_j, L)$ , by contracting  $y_j T y$ , and adding the valid sequence  $Cx$ . Notice this forms another cycle  $x_i C x$ . Since both cycles unload the same amount of jobs, when we contract  $y_j T y$ , the schedule has  $\text{jobs}(y_j T y)$  left to unload. These jobs are unloaded by the newly formed cycle  $x_i C x$  (since both  $x_i C x, y_j T y$  unload the same amount of jobs). Due to additivity property of  $\text{cost}()$ , the costs of both schedules can be written as follows:

$$\text{cost}(\Gamma) = \text{cost}((J, x_i C x, K, y_j, L)) + \text{cost}(y_j T y) \quad (13)$$

$$\text{cost}(\Gamma^*) = \text{cost}((J, x_i C x, K, y_j, L)) + \text{cost}(x_i C x) \quad (14)$$

We know  $\Gamma^*$  is a schedule that unloads the same amount of jobs as  $\Gamma$ , and  $\text{cost}(\Gamma^*) \leq \text{cost}(\Gamma)$ .

**Case 2:**  $\Gamma = (J, x_i C x T x, K)$ , where  $J, K$  are arbitrary sub-schedules. In this case  $x = y$ , but  $C \neq T$ . We can apply the same reasoning as in **Case 1**, and create a schedule  $\Gamma^*$  of the form  $(J, x_i C x C x, K)$  with a lower cost.  $\square$

## VII. 2-STATION OPTIMAL SCHEDULE

Lemma 1 implies a specific structure of an existing optimal schedule. We will exploit this to find *OS* for the 2-station system. Observing the 2-station system and all its possible states shown in Figure 3, it seems only two pure cycles can appear, and both unload exactly one job.

To verify this, notice that in the state (3), the AGV has no option and must transport the job from station 0 to station 1. It

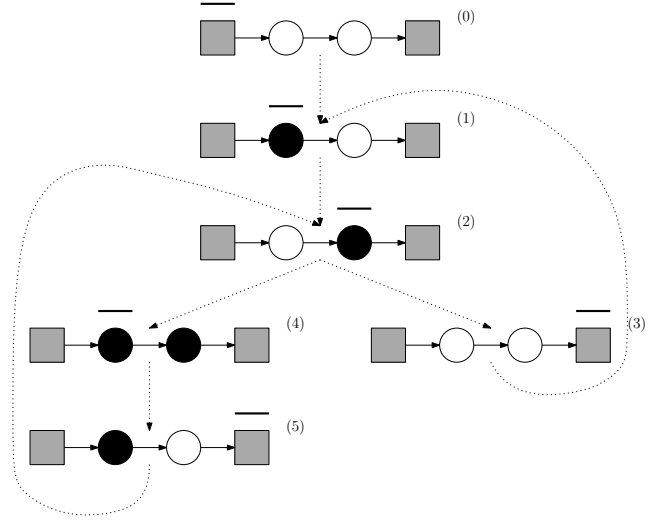


Fig. 3: All possible sequences of states in any schedule for a 2-station system. We do not depict the remaining processing times as they do not play a relevant role in this case.

is easy to check that all the remaining processing times, jobs, and AGV locations after this transition are equivalent to the state (1), hence there is an arrow pointing from (3) back to (1). The same reasoning is applied when considering all possible transportation tasks at state (5).

**Theorem 5.** *For  $m = 2$ , there exists an optimal schedule that has one of the two forms:*

$$(0, (1C1)^{*[n-1]}, 2, 3) \quad (15)$$

$$(0, 1, (2T2)^{*[n-1]}, 3) \quad (16)$$

Where states (0), (1), (2), (3), (4), (5) are defined in Figure 3,  $(2T2) = (2, 4, 5, 2)$ , and  $(1C1) = (1, 2, 3, 1)$ . Notation  $(aCa)^{*[k]}$  denotes that the cycle  $aCa$  appears  $k$  times, forming a valid sequence  $(aCaCa \dots aCaCa)$ . Additionally,  $(aCa)^{*[0]} = (a)$ .

*Proof.* Consider any other schedule other than the one defined in (15), and (16). When this schedule, schedule  $\Gamma$ , is inevitably at state (2), at one point state (3) follows, and at another point state (4) must follow (else we are back at one of the schedules in (15),(16)). Unless there are no jobs left to unload and the schedule stops at (3), Figure 3 implies that cycles (2, 4, 5, 2) and (2, 3, 1, 2) must appear. They must also be disjoint in  $\Gamma$  because as shown in Figure 3, once state (3) follows after state (2), the whole cycle must be completed (since the AGV has no other options) before the other cycle appears. Therefore, Lemma 1 applies.

To conclude, all schedules apart from (15) and (16) have two different disjoint indexed cycles in them, both unloading one job. Lemma 1 implies there is another schedule, which does not have two disjoint indexed cycles, with lower or equal makespan. There are only two schedules with these properties,

the schedules in (15) or (16), hence one of them must be an optimal schedule.  $\square$

**Corollary 1.** *Since cost is additive, we can decompose the total cost into two parts. In both forms – (15), (16), the cost((0,1,2,3)) is present. The only difference is the cost of the cyclic sub-schedule  $(1C1)^{*[n-1]}$  or  $(2T2)^{*[n-1]}$ . Thus if  $\text{cost}(1C1) \leq \text{cost}(2T2)$ , then the form in (15) is an optimal schedule and vice versa.*

Clearly, we can compute  $\text{cost}(1C1)$  and  $\text{cost}(2T2)$  in constant time w.r.t.  $n$  (since each of the cycles  $1C1$  and  $2T2$  have constant size) and thus we can solve the 2-station problem in constant time (if the output is given in the compressed format as in (15) and (16), else in time  $O(n \cdot (m+1))$ ).

To conclude, if the problem at hand concerns only two stations, it is possible to use BFS to form a graph as seen in Figure 3, determine the cheaper of the two pure cycles, and set the schedule of the AGV to be either (15), or (16), ensuring the lowest makespan.

### VIII. M-STATION CHEAP CYCLE SCHEDULE

We have seen that in the 2-station case, using the cheapest cycle in  $\text{cost}$  yields an optimal schedule. In  $m$ -station case, it is not as straightforward since a cycle can have low  $\text{cost}$  merely because it unloads a small number of jobs. Therefore, we will concern ourselves with  $\text{rate}$  instead of  $\text{cost}$ .

Considering these limitations, we will now inspect a generalization of the schedule proposed in the last section, the so-called *Cheap Cycle Schedule (CCS)*.

**Definition 6. Cheap Cycle Schedule (CCS)** *Let  $x^*C^*x^*$  be a cycle that can appear in a schedule of the  $m$ -station system we consider. Let  $\text{rate}(x^*C^*x^*)$  be the lowest rate of all possible cycles that can appear in a schedule of the system. The Cheap Cycle schedule is then*

$$CCS = (A, (x^*C^*x^*)^{*[r]}, B) \quad (17)$$

, where  $n - \text{jobs}(x^*C^*x^*) < r \leq \text{jobs}(x^*C^*x^*)$ , and  $A$  and  $B$  are sub-schedules that ensure  $CCS$  is a valid schedule.

We will show the following

**Theorem 6.**  $\lim_{n \rightarrow \infty} \frac{\text{makespan}(CCS)}{\text{makespan}(OS)} = 1$

Before proving the theorem, we will analyze the structure of an optimal schedule and the cycle with the lowest rate  $C^{x^*}$ .

**Definition 7. Set of disjoint indexed cycles** *is a set of indexed cycles that are mutually disjoint.*

**Lemma 2.** *All cycles in the set of disjoint indexed cycles  $M$  can be simultaneously contracted in  $\Gamma$ , resulting in the schedule  $\Gamma'$ .*

*Proof.* Suppose that we contract an arbitrarily  $x_i C x_i \in M$  on the schedule  $\Gamma = (\dots, x_i C x_i, \dots)$ . This will result in  $\Gamma^1 = (\dots, x_i, \dots)$ . Next, consider any other cycle  $y_j T y_j \in M$ . Since

$x_i C x_i$  and  $y_j T y_j$  are disjoint indexed, the latter cycle is still fully present in  $\Gamma^1$  (possibly at a different index now) and thus can be contracted. In this manner, we contract all cycles of  $M$  on  $\Gamma$  one by one.  $\square$

**Definition 8. Maximum Set of disjoint indexed cycles** *is a set of disjoint indexed cycles  $M^*$  in schedule  $\Gamma$  such that, after contracting all cycles from  $M^*$  in  $\Gamma$ , the resulting schedule  $\Gamma^*$  is the shortest in the amount of states. In other words, there does not exist another set of disjoint indexed cycles  $M'$  on the schedule  $\Gamma$ , such that, after contracting all cycles in  $M'$  resulting in schedule  $\Gamma'$ ,  $\text{Len}(\Gamma') < \text{Len}(\Gamma^*)$ . In this case  $\text{Len}()$  denotes the amount of states in a schedule.*

Trivially, a maximum set of disjoint indexed cycles must exist for any schedule.

**Lemma 3.** *Let  $M^*$  be a Maximum set of disjoint indexed cycles in schedule  $\Gamma$ . If every cycle in  $M^*$  is contracted, the resulting schedule  $\Gamma^*$  has  $\text{Len}(\Gamma^*) \leq (m+2) \cdot (\max_i p_i)^m \cdot 2^m$  (the upper bound on the number of unique states in (1)).*

*Proof.* Assume  $\text{Len}(\Gamma^*) > (m+2) \cdot (\max_i p_i)^m \cdot 2^m$ , then there exists a state  $z$  that must repeat at least twice in  $\Gamma^*$ . If  $z$  repeats twice in  $\Gamma^*$ , it also repeats twice in  $\Gamma$ , forming an indexed cycle  $z G z$  in the original schedule  $\Gamma$ .

Suppose, we create set  $M^{**}$  by adding  $z G z$  into  $M^*$ , and removing all cycles from  $M^*$  that are not disjoint to  $z G z$ . This ensures  $M^{**}$  is a set of disjoint indexed cycles, Lemma 2 applies, and we contract all cycles of  $M^{**}$  resulting in the schedule  $\Gamma^{**}$ .

Cycles of  $M^*$  that are **not** disjoint to  $z G z$  are proper sub-schedules of  $z G z$  – all of their indexed states also appear in  $z G z$ . For the sake of argument, suppose that  $d H d$  is a cycle of  $M^*$  that is not a proper subschedule of  $z G z$ . W.l.o.g. assume the original schedule to look as follows:  $(\dots, z_v, \dots, d_h, \dots, z_{v1}, \dots, d_{h1}, \dots)$ . But then  $z_{v1}$  is inside the inner part of  $d H d$ , which means  $z_{v1}$  would be contracted, not present in  $\Gamma^*$ , which cannot happen because  $z_{v1}$  is found in  $\Gamma^*$  (although at a different index).

When contracting cycles of  $M^*$  on  $\Gamma$ , the sub-schedule (and also a cycle)  $z G z$  is contracted to have form  $z_i \dots z_j$  in  $\Gamma^*$ ;  $z$  appears twice in  $\Gamma^*$  by assumption. When contracting cycles of  $M^{**}$ , the whole cycle  $z G z$  is contracted, resulting in just one state  $z_k$  in schedule  $\Gamma^{**}$ .

The schedule  $\Gamma$  outside of the sub-schedule  $z G z$  is the same after contracting cycles of  $M^*$  or  $M^{**}$ . This is trivial because cycles of  $M^*$  in outside of  $z G z$  are by definition disjoint to this indexed cycle, hence are also present in  $M^{**}$ . Therefore, the schedule  $\Gamma^{**}$  is at least one indexed state shorter (the state  $z_j$ ) than  $\Gamma^*$ .

This is a contradiction since  $M^*$  is the maximum set, therefore  $\text{Len}(\Gamma^{**}) \leq (m+2) \cdot (\max_i p_i)^m \cdot 2^m$ .  $\square$

**Corollary 2.** *Due to Lemma 3, every schedule  $\Gamma$  is made up of disjoint indexed cycles that span the entire schedule (the*

set  $M^*$ ), apart from the number of states bounded by (1). The sub-schedules of  $\Gamma$  that are not part of the set  $M^*$  define the same transportation tasks as the contracted schedule  $\Gamma^*$ .

We now consider a cycle with the lowest *rate*. A catch is that it is not obvious that a cycle of the lowest *rate* exists regardless of what  $n$  is. One could imagine that, as  $n$  grows arbitrarily large, larger and larger cycles can be formed with smaller and smaller rates. This would effectively disallow finding the cycle with the overall lowest *rate*. In the following Lemma, we show that there exists a cycle  $\mathbf{C}^{x^*}$  with the lowest *rate*, such that it is a pure cycle. A pure cycle is a cycle that does not contain any other cycle (i.e., no two states but the first and last are the same), thus has length at most  $(m+2) \cdot (\max_i p_i)^m \cdot 2^m$ . Otherwise, states start repeating inside the cycle forming smaller cycles. This ensures that no matter how many jobs the system unloads, the pure cycle  $\mathbf{C}^{x^*}$  will always have the lowest *rate*.

**Lemma 4.** *There exists a pure cycle  $\mathbf{C}^{x^*}$ , such that  $\text{rate}(\mathbf{C}^{x^*}) \leq \text{rate}(\mathbf{C})$ , where  $\mathbf{C}$  is any other cycle that can appear in a schedule for the considered  $m$ -station system.*

*Proof.* Let  $\mathbf{C}^{x^*}$  be a pure cycle with the lowest *rate* out of all pure cycles. Since pure cycles are bounded in length, there is only a finite number of them, hence  $\mathbf{C}^{x^*}$  exists. Consider any composite cycle  $\mathbf{C} = x \dots y T y \dots x$ . Let  $\mathbf{C}^x = x \dots y \dots x$  – the cycle after contracting  $\mathbf{T}^y$ . The *rate* of  $\mathbf{C}$  is (using the fact that both *cost* and *jobs* are additive):

$$\begin{aligned} \text{rate}(\mathbf{C}) &= \frac{\text{cost}(\mathbf{C}^x) + \text{cost}(\mathbf{T}^y)}{\text{jobs}(\mathbf{C}^x) + \text{jobs}(\mathbf{T}^y)} = \\ &= \text{rate}(\mathbf{C}^x) \cdot \frac{\text{jobs}(\mathbf{C}^x)}{\text{jobs}(\mathbf{C}^x) + \text{jobs}(\mathbf{T}^y)} + \\ &+ \text{rate}(\mathbf{T}^y) \cdot \frac{\text{jobs}(\mathbf{T}^y)}{\text{jobs}(\mathbf{C}^x) + \text{jobs}(\mathbf{T}^y)} = \\ &= \text{rate}(\mathbf{C}^x) \cdot r + \text{rate}(\mathbf{T}^y) \cdot (1-r) \quad r \in (0,1) \end{aligned} \quad (18)$$

Going from the first to the second line of (18), we use the fact that  $\text{cost}(\mathbf{C}^z) = \text{rate}(\mathbf{C}^z) \cdot \text{jobs}(\mathbf{C}^z)$  for any  $\mathbf{C}^z$ . The quantity in the above equation is a weighted average of  $\text{rate}(\mathbf{C}^x)$  and  $\text{rate}(\mathbf{T}^y)$ , thus  $\mathbf{C}$  does not have the lowest rate. Consider the inner cycle with the rate lower or equal to  $\text{rate}(\mathbf{C})$ , if it is pure, then by assumption  $\text{rate}(\mathbf{C}^{x^*})$  is smaller than the *rate* of the inner cycle. If the inner cycle is not pure, we apply the same reasoning again.  $\square$

*Proof of Theorem 6.* Let  $OS$  be an optimal schedule, and  $OS^*$  be the schedule after contracting all cycles in  $M^*$  of  $OS$ . Due to Corollary 2, the *cost* of  $OS$  can be written as follows:

$$\begin{aligned} \text{cost}(OS) &= \text{cost}(OS^*) + \sum_{\mathbf{C}^x \in M^*} \text{cost}(\mathbf{C}^x) = \\ &= \text{cost}(OS^*) + \sum_{\mathbf{C}^x \in M^*} \text{jobs}(\mathbf{C}^x) \cdot \text{rate}(\mathbf{C}^x) \end{aligned} \quad (19)$$

Now, we also decompose the *cost* of  $CCS$  (defined in (17)), into parts. Let  $\tilde{n} = \sum_{\mathbf{C}^x \in M^*} \text{jobs}(\mathbf{C}^x)$  be total amount of jobs unloaded in cycles of  $M^*$ , and let  $n' = n - (\text{jobs}(B) + \text{jobs}(A))$  be the total amount of jobs unloaded by  $\mathbf{C}^{x^*}$  in  $CCS$  ( $A, B$  are of length independent of  $n$  – Definition 6).

Let:

$$\begin{aligned} s &= \max_a (a \cdot \text{jobs}(\mathbf{C}^{x^*})) \\ \text{s.t. :} \\ a \cdot \text{jobs}(\mathbf{C}^{x^*}) &\leq \tilde{n} \\ a \cdot \text{jobs}(\mathbf{C}^{x^*}) &\leq n' \end{aligned} \quad (20)$$

We know  $n$  differs from  $\tilde{n}$  only by the constant  $\text{jobs}(OS^*)$  due to Corollary 2, and from  $n'$  by a constant due to Definition 6. Lemma 4 states that no matter how large  $n$  is, the cycle with lowest *rate* remains the same, thus  $\text{jobs}(\mathbf{C}^{x^*})$  is constant. Putting this all together,  $\min(\tilde{n}, n') - \text{jobs}(\mathbf{C}^{x^*})$  differs from  $n$  only by a constant.

Trivially,  $n \geq s \geq \min(\tilde{n}, n') - \text{jobs}(\mathbf{C}^{x^*})$  because of (20), thus if  $n \rightarrow \infty$ , then  $s \rightarrow \infty$ .

Let  $k = n - s - (\text{jobs}(A) + \text{jobs}(B))$  be the rest of the jobs that are unloaded via  $\mathbf{C}^{x^*}$  in  $CCS$ . Clearly,  $k$  does not grow to infinity when  $n$  does, because  $s$  differs from  $n$  only by a constant. The *cost* of  $CCS$  can be written as follows:

$$\text{cost}(CCS) = \text{cost}(A) + (k+s) \cdot \text{rate}(\mathbf{C}^{x^*}) + \text{cost}(B) \quad (21)$$

The makespan ratio of the two schedules is the following:

$$\begin{aligned} \frac{\text{makespan}(CCS)}{\text{makespan}(OS)} &= \frac{\text{cost}(CCS)}{\text{cost}(OS)} = \\ &= \frac{\text{cost}(A) + (k+s) \cdot \text{rate}(\mathbf{C}^{x^*}) + \text{cost}(B)}{\text{cost}(OS') + \sum_{\mathbf{C}^x \in M^*} \text{jobs}(\mathbf{C}^x) \cdot \text{rate}(\mathbf{C}^x)} \leq \\ &= \frac{\text{cost}(A) + (k+s) \cdot \text{rate}(\mathbf{C}^{x^*}) + \text{cost}(B)}{\text{cost}(OS') + s \cdot \text{rate}(\mathbf{C}^{x^*})} \leq \\ &= \frac{\text{cost}(A) + (k+s) \cdot \text{rate}(\mathbf{C}^{x^*}) + \text{cost}(B)}{s \cdot \text{rate}(\mathbf{C}^{x^*})} = \\ &= 1 + \frac{\text{cost}(A) + \text{cost}(B) + k \cdot \text{rate}(\mathbf{C}^{x^*})}{s \cdot \text{rate}(\mathbf{C}^{x^*})} \xrightarrow{n \rightarrow \infty} 1 \end{aligned} \quad (22)$$

In (22), going from the second to third line, we know  $\sum_{\mathbf{C}^x \in M^*} \text{jobs}(\mathbf{C}^x) \cdot \text{rate}(\mathbf{C}^x) \geq s \cdot \text{rate}(\mathbf{C}^{x^*})$ . This is due to the fact that  $\mathbf{C}^{x^*}$  always has the lowest *rate*, and by (20),  $s$  is smaller or equal to the total amount of jobs unloaded by the cycles of  $M^*$ . In the penultimate line of (22), all quantities in the fraction numerator are finite, hence the fraction nears zero for large  $n$ .  $\square$

Theorem 6 says there is an almost optimal schedule, described in Definition 6. The only components needed to construct it are  $x^* C^* x^*$ ,  $A, B$  and  $r$ . To compute this schedule, we build a graph as in Figure 3, simulating all possible

transportation tasks the AGV can do, and the states they result in. Edge costs are defined as the time it takes to execute a transportation task. Next, we enumerate all pure cycles in the graph and find the one with the lowest *rate*, which by Lemma 4 has *lowest* *rate* irrespective of  $n$  (if there are multiple such cycles, we pick the one that unloads the least number of jobs). Finally, we find sub-schedule  $A$ , such that  $A = (x_0, \dots, x^*)$ , where  $x^*$  is arbitrary state in the cycle  $x^*C^*x^*$ . The value  $r$  is the largest integer such that  $r \cdot \text{jobs}(C^*) \leq n - \text{jobs}(A)$ . We find  $B$  such that  $B = (x^*, \dots, x_{n \cdot (m+1)})$ . The sequence

$(A, \overbrace{C^*x^*C^*, \dots, C^*x^*C^*}^{r\text{-times}}, B)$  is a valid schedule that unloads  $n$  jobs. The sub-schedules  $A$  and  $B$  can be found by any shortest path algorithm, for instance, Dijkstra [15].

#### A. Cycle with the lowest rate in optimal schedule

So far, it is unknown if a cycle with the lowest *rate* directly appears in  $OS$ . If it does, it can provide more insight on how  $OS$  might look, and how different it is compared to  $CCS$ .

Let the *cost* of the constant part of  $CCS$  be  $\alpha$ , and the *cost* of the constant part of  $OS$  be  $\beta$ . Furthermore, let  $C'$  be a pure cycle, with the *second* lowest *rate*. It is unknown whether the cycles in the maximum set of disjoint indexed cycles  $M^*$  are pure or composite. Nevertheless, *cost* of any composite cycle can be written as the sum of *costs* of the pure cycles it is composed of (similar to how its *rate* can be decomposed, shown in Lemma 4). Let  $C^x$  be a composite cycle, then  $\text{cost}(C^x) = \sum_{C^y \in C^x} \text{cost}(C^y)$ , where  $C^y$  denotes every pure cycle, the cycle  $C^x$  is composed of.

We can now compare the *cost* of  $OS$  and  $CCS$  as follows:

$$\begin{aligned} \text{cost}(CCS) - \text{cost}(OS) &= \\ (\alpha + s \cdot \text{rate}(C^*)) - (\beta + \sum_{C^x \in M^*} \text{cost}(C^x)) &= \\ \gamma + (s \cdot \text{rate}(C^*)) - (\sum_{C^x \in M^*} \sum_{C^y \in C^x} \text{jobs}(C^y) \cdot \text{rate}(C^y)) &\leq \\ \gamma + (s \cdot \text{rate}(C^*)) - (s \cdot \sum_{C^x \in M^*} \sum_{C^y \in C^x} \text{rate}(C^y)) &\stackrel{s=\theta+s'}{\leq} \\ \gamma + ((\theta + s') \cdot \text{rate}(C^*)) - (s' \cdot \text{rate}(C^*) + \theta \cdot \text{rate}(C')) &= \\ \gamma + \theta \cdot (\text{rate}(C^*) - \text{rate}(C')) & \end{aligned} \quad (23)$$

Going from the second to the third line of (23), we break up the costs of cycles in  $M^*$  into the costs of the pure cycles they are composed of – denoted in the double summation. In (20) we constrained  $s$  to be at most the number of jobs unloaded by the cycles in  $M^*$ , hence from the third to fourth line of (23) we can substitute the summation of jobs by  $s$ .

The *rates* of all the pure cycles are higher or equal than  $\text{rate}(C^*)$ . Possibly, some of the pure cycles may have *rate* strictly higher than  $\text{rate}(C^*)$ . This *rate* is at least  $\text{rate}(C')$ . To account for this, we split up the  $s$  jobs into  $s'$  which are unloaded with at least the lowest *rate*, and  $\theta$  jobs which are unloaded with at least the **second** lowest *rate*. This split-up

models the possibility that some of the pure cycles, the cycles of  $M^*$  are composed of, do not have the lowest *rate*.

In the last line of (23), the quantity  $(\text{rate}(C^*) - \text{rate}(C'))$  is, by definition, a negative constant. But then, if  $\theta$  is too large, the whole quantity (23) will eventually become negative which is a contradiction, since  $OS$ , by definition, cannot have a larger makespan than  $CCS$ . Therefore,  $\theta$  is bounded from above.

This implies that the composite cycles in  $M^*$  of  $OS$  must be composed of pure cycles with the lowest *rate*. There may be some pure cycles (which the composite cycles are composed of) that have a higher *rate*, but these cycles can unload at most  $\theta$  jobs, else  $OS$  does not have the lowest makespan which is a contradiction.

To conclude, any  $OS$  is made of the cycles in  $M^*$ , and the contracted schedule  $OS^*$  which is finite. Additionally, the cycles in  $M^*$  must be composed of pure cycles that unload  $(n - \text{jobs}(OS^*) - \theta)$  jobs, and have the lowest *rate*. If there is only one unique pure cycle with the lowest *rate*, it implies that following the schedule  $OS$ , any  $m$ -station system with an AGV will find itself in the same states, performing the same transportation tasks as when following  $CCS$ .

## IX. DISCUSSION

Generally speaking, if the travel times between stations are larger than the processing times, it may be sufficient to simply use the primitive schedule irrespective of the number of stations or jobs. In a real case scenario, there is a lot of flexibility when arranging stations, thus also the travel times. This cannot be done with processing times, since these are given by the ability of the machine to process a job within a given time frame. Therefore, the case  $P > D$  is more relevant, where we have found the makespan ratio of  $PS$  and  $OS$  to scale linearly with  $m$ . There is a very sensible interpretation since the advantage of a good schedule should be the parallel processing of  $m$  jobs in the  $m$  stations. The primitive schedule is missing exactly that, processing only one job at a time. Reductions seem very useful since they enable us to study large systems via simple ones. We used them to tighten the upper bound on the makespan ratio of the primitive and optimal schedule to be within  $m$  and  $m + 2$ . This paper didn't explore any other reductions. One potential use could be reducing a problem from an  $m$ -station system to a 2-station system, which could provide a stricter lower bound for the makespan. Other overlooked reductions could enhance our understanding of makespan or the overall problem mechanics.

The Theorem 6 is in theory very exciting, however in practice it has its problems. For instance, if the number of jobs has to be in the order of  $10^{100}$  for the makespan ratio of  $CCS$  and  $OS$  to near 1, it is of no use to us. If 2-approximation schedule is sufficient, (22) suggests that  $n$  has to grow exponentially w.r.t.  $m$ . This is because the finite part present at the numerator is bounded from above in the number of states with Quantity (1) which is exponential in  $m$ . Of course, this does not mean that the finite part of  $CCS$  will

grow exponentially w.r.t.  $m$ , it merely means its upper bound is exponential. It would be relevant to know how exactly this ratio evolves with growing  $n$ , which could be studied experimentally.

It is unknown whether  $OS$  is unique, but the results in the above sections indicate the existence of at least one very structured  $OS$ . We know that only a finite part of this  $OS$  is not comprised of cycles. Additionally, the cycles that make up the schedule  $OS$  must be composed of pure cycles with the lowest *rate*. Other results restrict this  $OS$  even further. For instance, two cycles that unload the same number of jobs cannot appear in  $OS$  simultaneously if they have different costs (Lemma 1). Finally, cycles in a schedule can be easily contracted or added while maintaining the same *cost*. Taking all this into consideration, there must be an  $OS$  which has a very restricted form. This  $OS$  may make most of the other schedules obsolete, and the task of finding a schedule with the lowest makespan will be reduced to analyzing a smaller set of schedules, similar to the 2-station case.

The problem in its current definition is very narrow, and certain extensions could be done. For instance, introducing multiple AGVs, and various types of jobs would still make the Quantity (1) constant w.r.t.  $n$ . There are limits however; for instance, if we introduce infinite buffers, or even worse, we allow rearranging jobs in the pick-up station, we make the states directly dependent on  $n$ , which would disallow for repeating states. No repeating states means no cycles in the schedule, which disrupts all theory we rely on. Finally, when proposing the method to find  $CCS$ , we assumed  $m$  to be a constant. However, the method for finding  $CCS$  is exponential in  $m$  because the number of possible states any system can be in is also exponential in  $m$ . At the moment, it is unknown whether there are ways to overcome this complexity hurdle. On the other hand, it is hard to imagine a real case scenario where  $m$  is so large that finding the  $CCS$  is intractable.

## X. CONCLUSION

We have shown that using reductions and direct analysis, we can derive lower bounds on any schedule for a system in a line layout. We have designed a primitive schedule, and shown that it is an optimal schedule if all distances are larger than all processing times in the system. In the general case, we have found the primitive schedule to be an  $m$ -approximation of an optimal schedule in the makespan.

Furthermore, we demonstrate that the described problem can be represented using states that form a sequence serving as a schedule for the AGV to follow. These states are finite in number and will repeat for large enough  $n$ , creating cycles. We exploit these cycles to define efficient schedules w.r.t. makespan.

For the case of 2-station, we have proven the existence of an optimal schedule comprising a single repeating cycle. By simulating AGV transportation tasks, we can determine the cycle with the lowest cost, enabling us to identify an optimal schedule for any number of jobs in a constant amount of time w.r.t. the number of jobs.

The idea is then extended on a general  $m$ -station case where we propose a so-called cheap cycle schedule that can be found in a constant time w.r.t. number of jobs. The makespan ratio of this and an optimal schedule approaches 1 as  $n$  grows to infinity. Further analysis demonstrates that the majority of the states and transportation tasks in an optimal schedule are also present in the cheap cycle schedule (provided the cycle with the lowest *rate* is unique), confirming their similarity both in costs and content.

For future research, it would be interesting to investigate other possible reductions and consider their implications on the original system. The problem is currently quite restricted in its setting, so it would be relevant to investigate how much we can relax the setting while still upholding the theory. Our results suggest a very strong structure of an existing optimal schedule. Formalizing the structure would grant better insight into what this optimal schedule looks like. This would enable a more detailed analysis of how the cheap cycle schedule compares to an optimal schedule for growing  $n$ . It could also serve as a robust tool if one aims to find an optimal schedule directly.

## REFERENCES

- [1] Jozef Grabowski and Jaroslaw Pempera. Sequencing of jobs in some production system. *European Journal of Operational Research*, 125(3):535–550, 2000.
- [2] Hanif D. Sherali, Subhash C. Sarin, and Murali S. Kodialam. Models and algorithms for a two-stage production process. *Production Planning & Control*, 1:27–39, 1990.
- [3] Robert J. Wittrock. Scheduling algorithms for flexible flow lines. *IBM Journal of Research and Development*, 29(4):401–412, 1985.
- [4] Rubén Ruiz and José Antonio Vázquez-Rodríguez. The hybrid flow shop scheduling problem. *European Journal of Operational Research*, 205(1):1–18, 2010.
- [5] A.S. Jain and S. Meeran. Deterministic job-shop scheduling: Past, present and future. *European Journal of Operational Research*, 113(2):390–434, 1999.
- [6] Hong Wang. Flexible flow shop scheduling: optimum, heuristics and artificial intelligence solutions. *Expert Systems*, 22(2):78–85, 2005.
- [7] Andreas Fink and Stefan Voß. Solving the continuous flow-shop scheduling problem by metaheuristics. *European Journal of Operational Research*, 151(2):400–414, 2003. Meta-heuristics in combinatorial optimization.
- [8] Mohamed Kharbeche, Jacques Carlier, Mohamed Haouari, and Aziz Moukrim. Exact methods for the robotic cell problem. *Flexible Services and Manufacturing Journal*, 23:242–261, 01 2010.
- [9] Erik Boom, Matúš Mihalák, Frank Thuijsman, and Mark H. M. Winands. Scheduling single agv in blocking flow-shop with identical jobs. In *Proceedings of the 13th International Conference on Operations Research and Enterprise Systems - ICORES*, pages 325–332. INSTICC, SciTePress, 2024.
- [10] Jacques Carlier, Mohamed Haouari, Mohamed Kharbeche, and Aziz Moukrim. An optimization-based heuristic for the robotic cell problem. *European Journal of Operational Research*, 202:636–645, 05 2010.
- [11] Mohamed Kharbeche, Jacques Carlier, Mohamed Haouari, and Aziz Moukrim. Exact methods for the robotic cell problem. *Flexible Services and Manufacturing Journal*, 23:242–261, 01 2010.
- [12] Johann L. Hurink and S. Knust. Makespan minimization for flow-shop problems with transportation times and a single robot. *Discrete applied mathematics*, 112(1-3):199–216, September 2001.
- [13] Nicholas G. Hall and Chelliah Sriskandarajah. A survey of machine scheduling problems with blocking and no-wait in process. *Operations Research*, 44(3):510–525, 1996.
- [14] M.H. Fazel Zarandi, H. Mosadegh, and M. Fattahi. Two-machine robotic cell scheduling problem with sequence-dependent setup times. *Computers Operations Research*, 40(5):1420–1434, 2013.
- [15] Edsger W Dijkstra. A note on two problems in connexion with graphs. *Numerische mathematik*, 1(1):269–271, 1959.